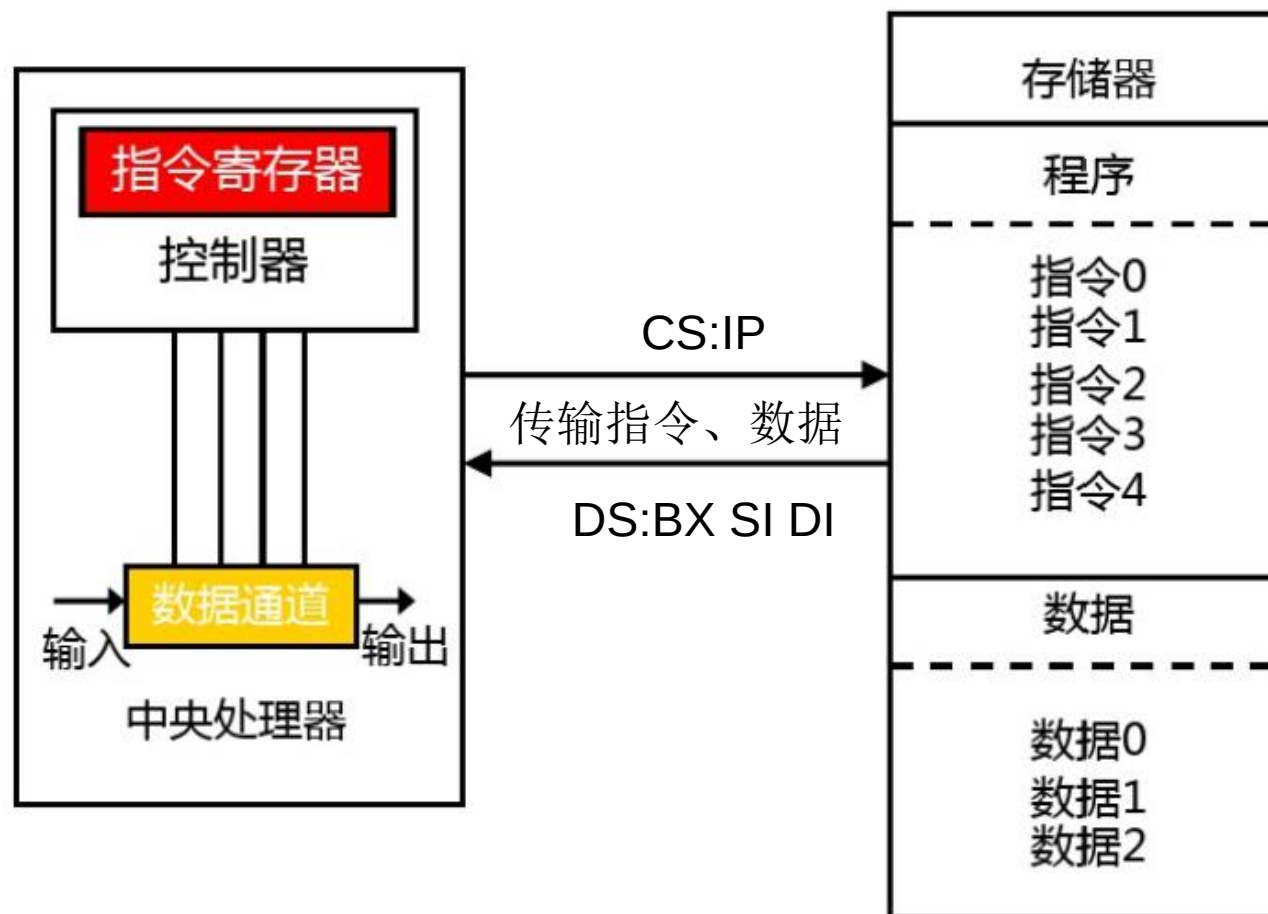


ARM 补充内容

Dr. Prof. Ni Li

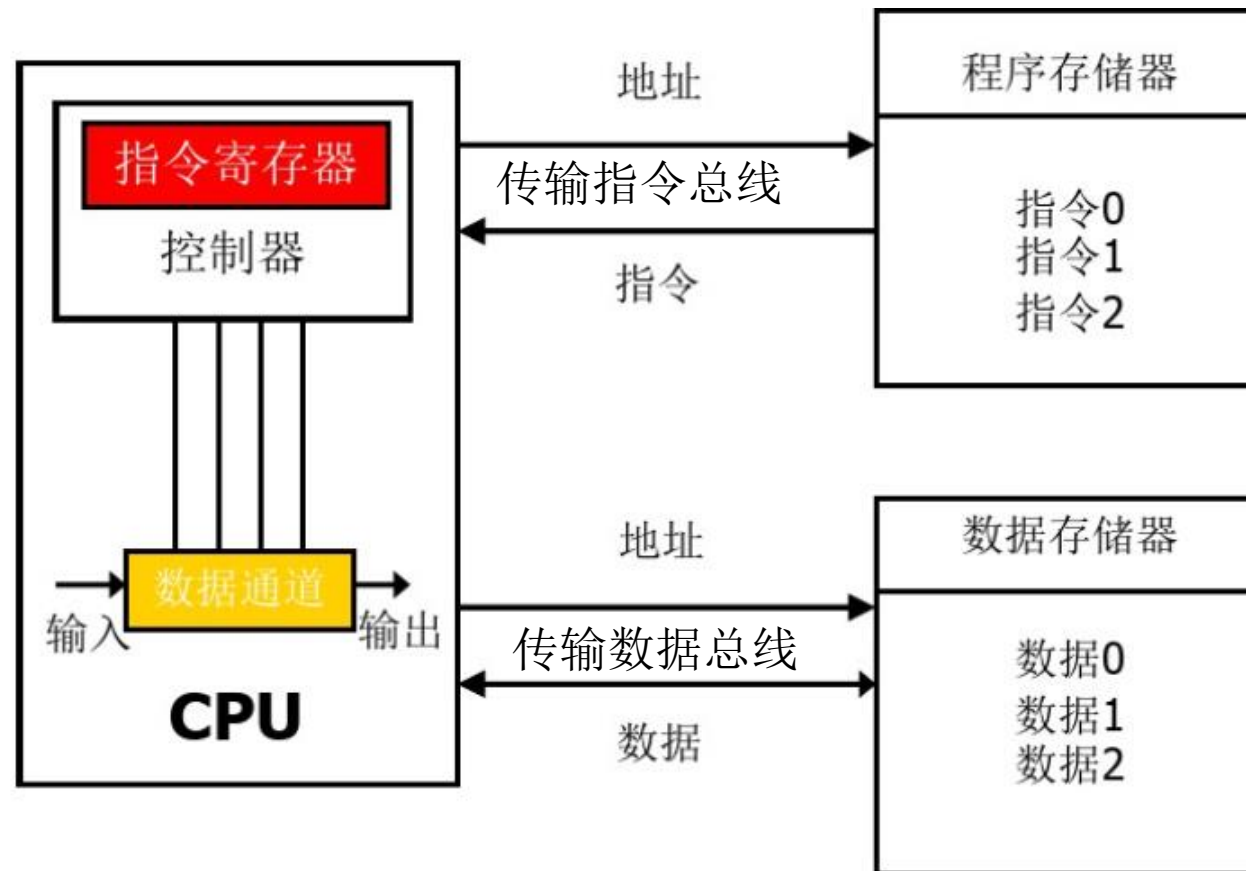
冯诺依曼架构----X86

- 程序+数据



哈佛架构---ARM

- 独立: Separated Program and Data storage



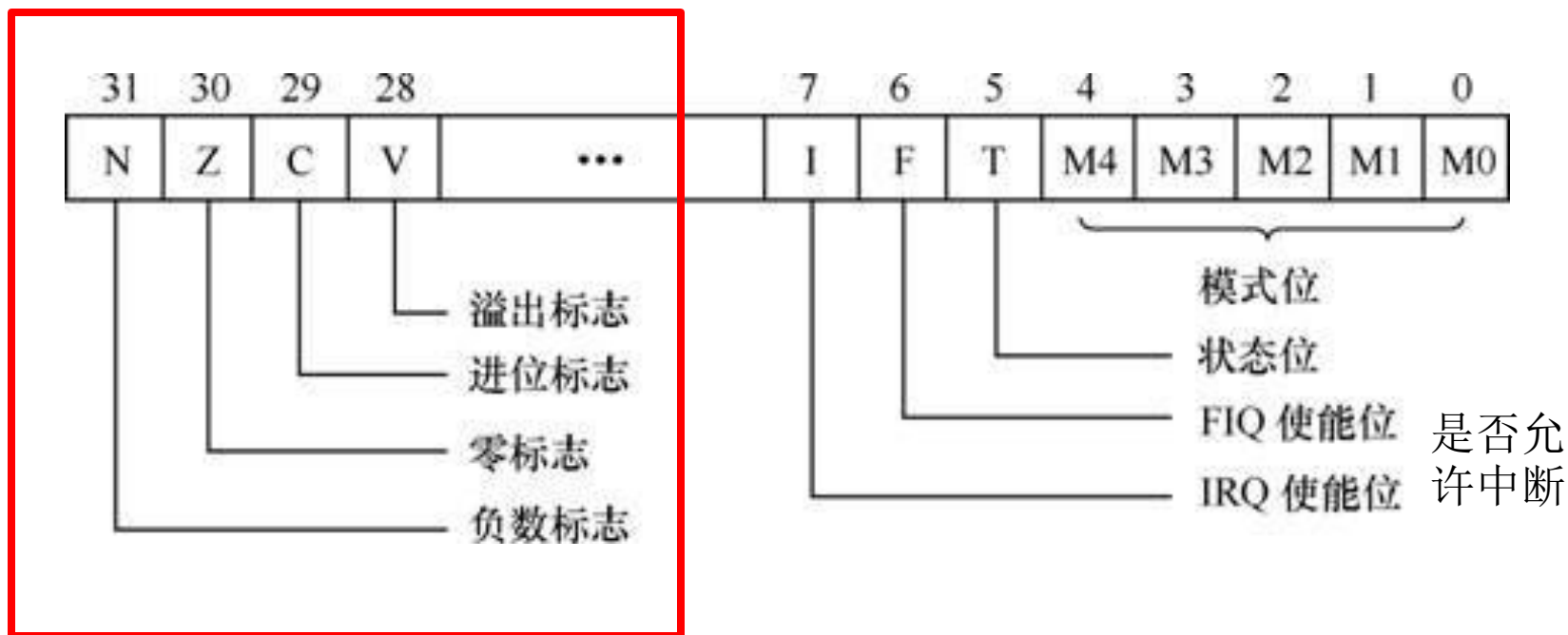
ARM 寄存器组

ADD AL, [100H]

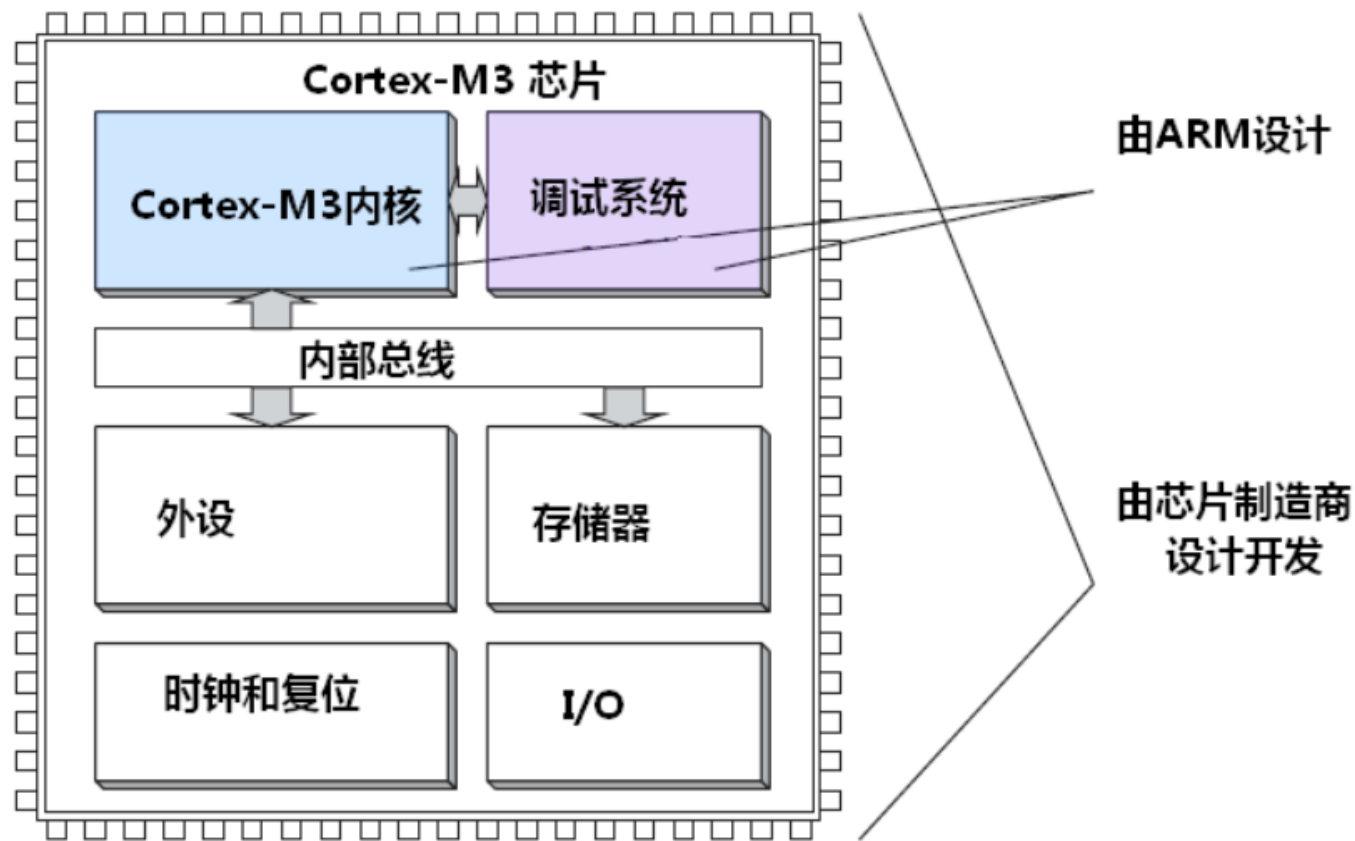
- ARM处理器共有**37**个寄存器
 - 采用RISC和哈佛架构，ALU不能直接访问内存，采用load和store指令实现寄存器和内存的数据交换，因此需要增加寄存器数量
- 32位寄存器
 - ◆ 31个通用寄存器，6个状态寄存器
 - ◆ ARM任意处理器模式下可用寄存器
 - 15个通用寄存器（R0~R14）
 - R13：堆栈指针→SS:SP
 - R14（LR寄存器）调用子程序时存储返回地址或响应异常/中断时存储断点地址→入栈保存
 - 程序计数器（PC）R15：指向下一条待执行指令地址
 - 状态寄存器CPSR

ARM 寄存器组

- CPSR (Current Program Status Register)



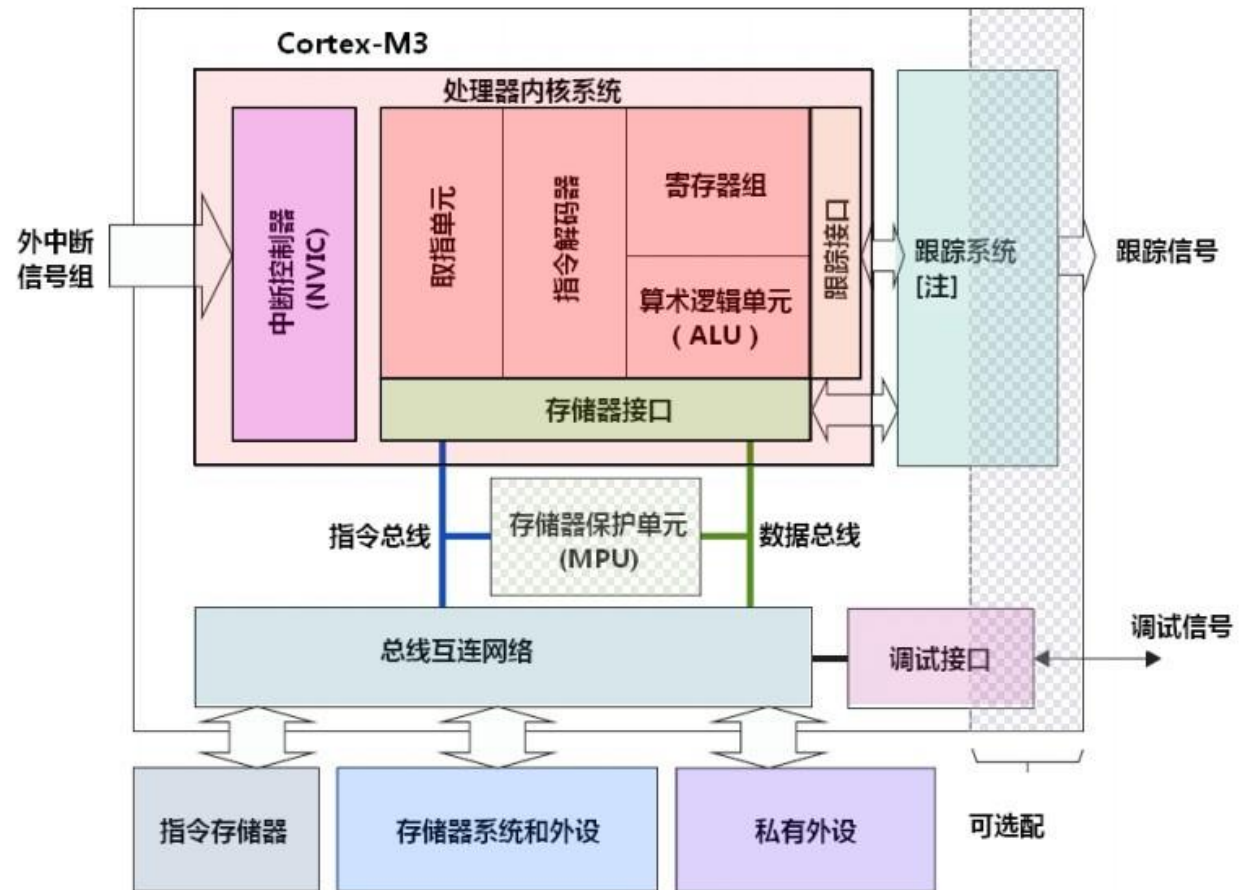
ARM系统组成



- 一般IO接口芯片包括GPIO（连接外设），UART（串口），定时器和AD/DA
- 具体几路AD，几路GPIO，几个定时器，连接哪些外设由各个厂商设计开发

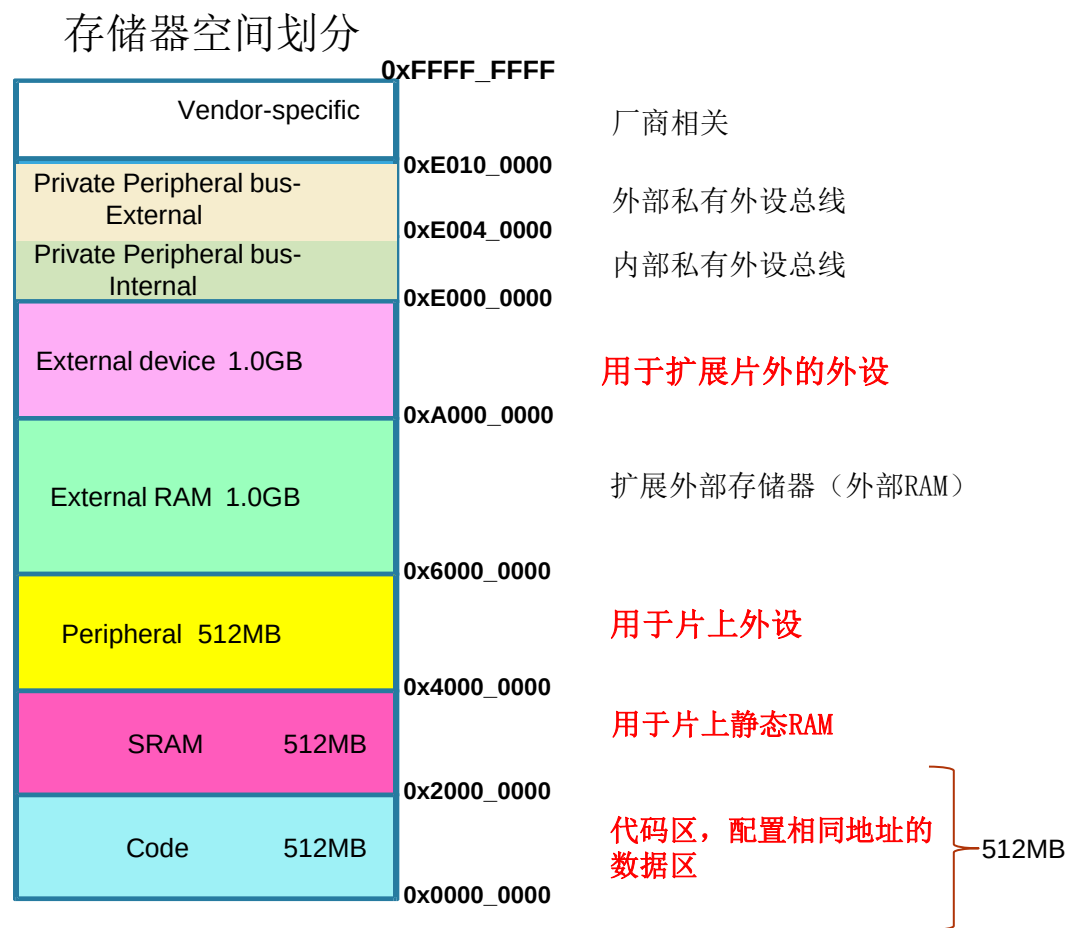
ARM Cortex-M3 内核--结构

- **哈佛结构**：独立的指令总线 and 数据总线，取指与数据访问并行工作
- **32位处理内核**：内部数据总线和寄存器为32位
- **存储器保护单元**：
 - 任务隔离
 - 关键数据区可读设置
 - 检测堆栈溢出，数组越界等
- **跟踪系统**
 - 指令运行历史记录
 - 内存信息
 - 性能分析：不同操作任务使用的时钟周期数



Cortex-CM3编址：存储器映射方式

- 地址空间：4GB 32位地址
- 程序可以在代码区，内部SRAM区以及外部RAM区执行
 - Code区/数据区，取指令通过I-Code总线，数据访问通过D-Code总线
 - SRAM区：片内RAM，由芯片厂商决定使用多大的内存，可读可写；当片内资源不够时，可有片外扩展RAM
 - 外设区，对外设寄存器地址进行映射，由芯片厂商布局不同的外部设备



X86 Vs. ARM

- CISC (Complex Instruction Set Computer)
 - **Variable** instruction length 可变指令长度
 - **Rich** instruction set, more **extensibility**
 - Direct access to memory locations 直接访问内存
 - 复杂电路设计(decoding, execution)
- RISC(Reduced Instruction Set Computer)
 - **Fixed** instruction length 固定指令长度
 - **Reduced** instruction set, 相对简单的设计
 - Load/store: 间接访问内存
 - 复杂指令由指令组合完成

X86 Vs. ARM

- 寄存器寻址

- MOV R0, #0xFF000; 将立即数0xFF000装入R0寄存器（立即数寻址、寄存器寻址）
- LDR R1, [R2]; 将R2指向的存储单元的数据读出，保存在R1中（源：寄存器间接寻址）
- LDR R2, [R3, #0X0C]; 读取R3+0X0C地址上的存储单元的内容，存入R2（源：寄存器相对寻址）
- STR R1, [R0]; 把R1的值保存到R0指定的存储单元（目的：寄存器间接寻址）

X86 Vs. ARM

内存单元: A B C

$$C=A+B$$

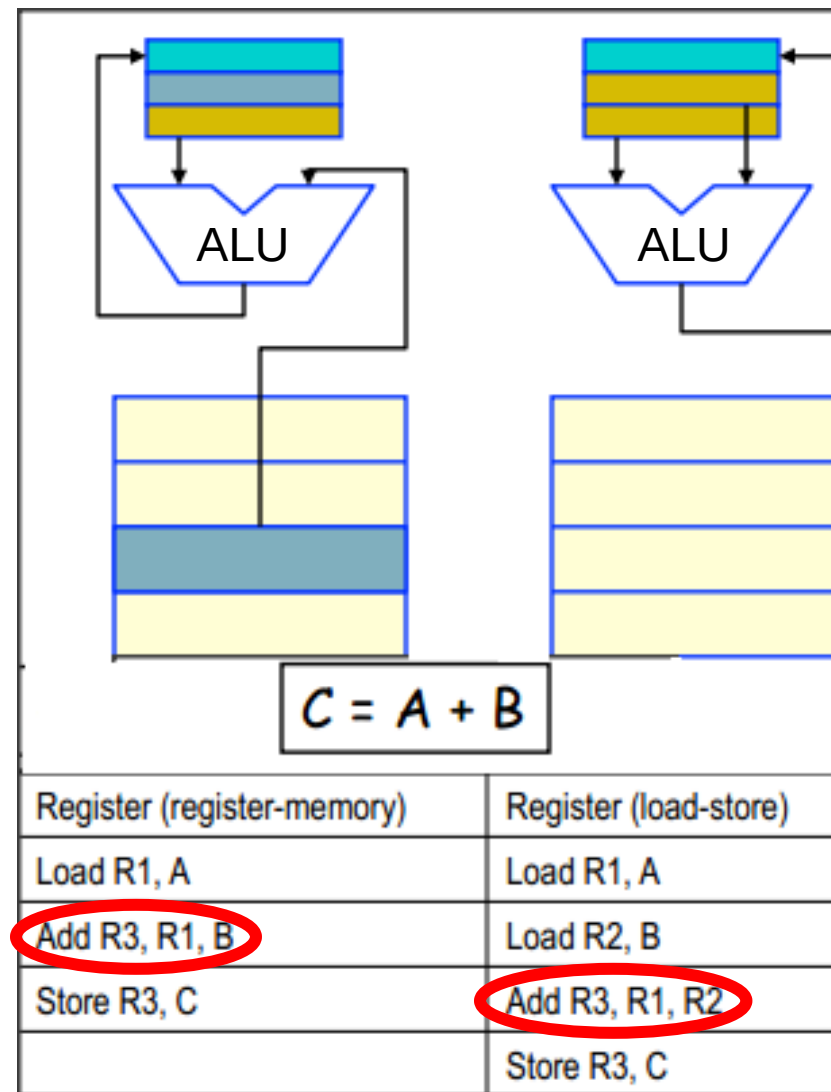
便捷接口

(X86: 直接访问存储单元)

Vs.

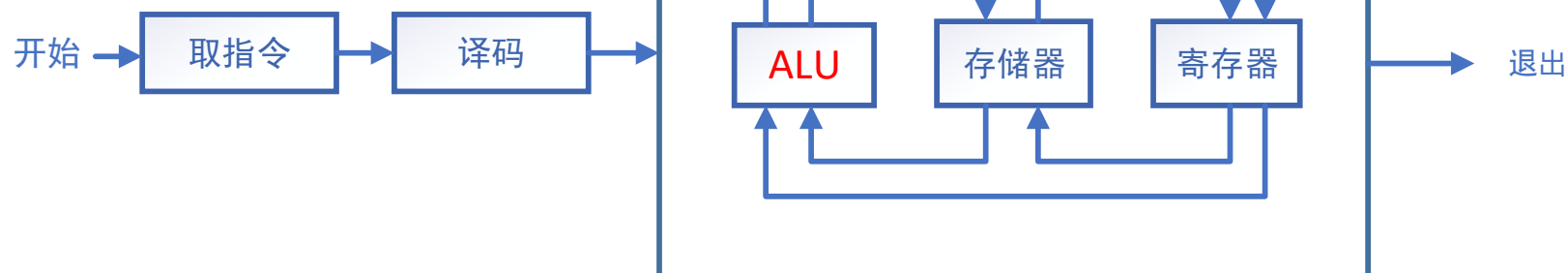
硬件执行效率

(ARM: 有限的指令功能简化底层硬件)



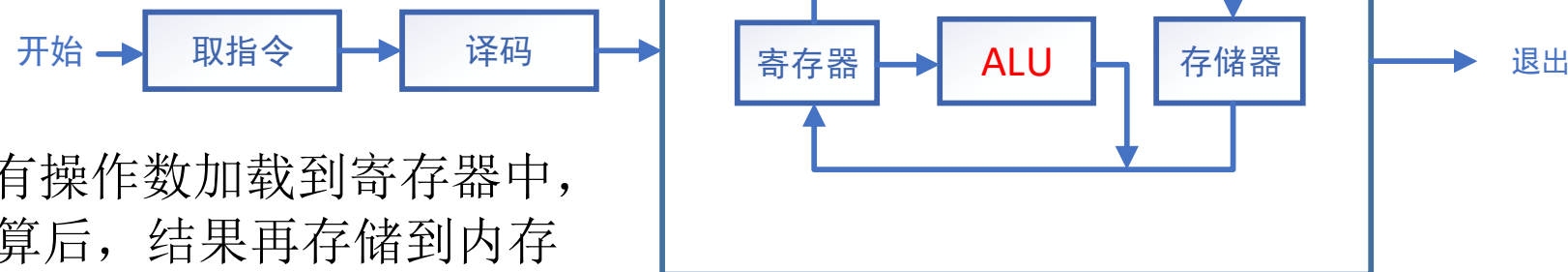
X86 Vs. ARM

- 数据传输路径



操作数可以是存储单元和寄存器，
更友好的指令调用

CISC: 寻址方式复杂



所有操作数加载到寄存器中，
完成计算后，结果再存储到内存
中。有限的指令功能带来简化的
底层硬件

RISC: Load/Store结构

X86 Vs. ARM

	CISC	RISC
指令集	Rich	Reduced, 通常<100
执行周期	执行指令时间较长 eg. 内存数据访问	执行指令时间较短
指令长度	可变长度 1-15 bytes	固定长度, 通常 4 bytes
寻址方式	多种寻址方式	相对简单
操作	对存储单元和寄存器进行算术和逻辑运算	对寄存器进行算术和逻辑运算 (Load/Store)

指令集

- 5 大类
 - 数据传送指令
 - 算术指令
 - 逻辑和移位指令
 - 控制转移指令
 - 串操作~~指令~~

数据传送指令

类型	X86	ARM
通用数据传送指令	MOV（传送） PUSH（进栈） POP（出栈） XCHG（交换）	MOV（立即数） LDR（单数据装载） STR（单数据存储） SWP（单一数据交换） LDM（多数据装载） STM（多数据存储）
访问外设专用指令	IN（输入指令） OUT（输出指令）	存储器映射 Memory-mapped
地址传送指令	LEA（取有效地址）	按照物理地址访问

- MOV R0, #0xFF000; 将立即数0xFF000装入R0寄存器
- LDR R2, [R3, #0X0C]; 读取R3+0X0C地址上的存储单元的内容，存入R2
- STR R1, [R0]; 把R1的值保存到R0指定的存储单元
- SWP R1, R2, [R3]; 将内存单元（R3）中的字数据读取到R1寄存器中，同时将R2寄存器的数据写入到内存单元（R3）中

算术指令

类型	X86	ARM
加法指令	ADD（加法） ADC（带进位加法） INC（加1）	ADD（加法） ADC（带进位加法） ADDS（影响标志位）
减法指令	SUB（减法） SBB（带借位减法） DEC（减1） NEG（取负） CMP（比较）	SUB（减法） SBC（带借位减法） SUBS（影响标志位） CMP（比较指令）

逻辑和移位指令

类型	X86	ARM
逻辑指令	AND（逻辑与） OR（逻辑或） NOT（逻辑非） XOR（异或） TEST（测试）	AND（逻辑与） BIC（清除位） EOR（逻辑异或） ORR（逻辑或）
移位指令	SHL（逻辑左移） SAL（算术左移） SHR（逻辑右移） SAR（算术右移） ROL（循环左移） ROR（循环右移） RCL（带进位循环左移） RCR（带进位右移）	LSL（逻辑或算术左移） ASL（逻辑或算术左移） LSR（逻辑右移） ASR（算术右移） ROR（循环右移） RRX（带扩展的循环右移）

控制转移指令

类型	X86	ARM
无条件转移指令	JMP（段间和段内转移）	B（跳转） 不区分短转移和近转移（指令长度一样）
条件转移指令	JZ（结果为0或相等则转移） JS（结果为负则转移） JO（溢出则转移）等等	BEQ（相等跳转） BNE（不等跳转）等等
子程序指令	CALL（调用指令） RET（返回指令）	BL（跳转，并保存下一条指令的地址到LR） MOV PC, LR（从函数返回）
循环指令	LOOP（循环指令） LOOPNZ/LOOPNE（当不为0或不相等时循环指令） LOOPE/LOOPNE（当为0或相等时循环指令）	ARM指令集没有X86那样专用的循环指令，这些指令都是通过B、BL指令结合其他的指令来实现的

控制转移指令

- Examples

B Label; 程序跳转到标号Label处执行

MOV R0, #loopcount; 初始化循环次数

loop: ;循环开始

...

SUBS R0, R0, #1; 循环计数器减1, 同时影响标志位

BNE loop; 如果R0不为0, 跳转到loop处继续执行

ARM 汇编程序

- ARM汇编语言程序与X86汇编程序类似：
 - 由指令和伪指令组成
 - 以程序段为单位组织代码。程序段分为代码段和数据段。代码段存放指令，数据段存放程序运行时所需的数据
 - 至少有一个代码段
- ARM没有x86中的段寄存器

例：

伪指令：段定义，只读

```
AREA Ex1, CODE, READONLY
```

```
ENTRY
```

伪指令：标识程序入口

```
MOV R0, #3FF5000
```

```
LDR R1, 0xFF
```

```
STR R1, [R0]
```

```
END
```

伪指令：
标识程序结束

指令

ARM 汇编程序

- 数据定义伪指令：DCB、DCW、DCD（类似X86汇编中的DB、DW、DD）以及SPACE（类似X86汇编中的 DB n DUP(0)）

- 例如：

```
AREA DATA1,DATA, READWRITE; 定义数据段，可读写
DT1  DCB  0xA5; 定义字节数据0xA5
DT2  DCB  "A Sample"; 定义字符串A Sample
DT3  DCW  0xFF12, 6 ;定义半字（16位）0xFF12和0x0006
DT4  DCD  0xFFFF1234; 定义字数据(32位)0xFFFF1234
DT5  SPACE 10; 分配连续10个字节存储单元并初始化为0
END
```

注：

ARM系统中字数据的字长是32位，而8086是16位

ARM 汇编程序示例

- 求两个数组DATA1和DATA2对应的数据之和，并把结果存入新数组SUM中。
- 相加过程中若结果为0则结束，并把新数组长度保存在R0寄存器中。

;定义数据段

AREA BlockData, DATA, READWRITE

DATA1 DCD 2,5,0,3,-4,5,0,10,9;定义字数据(32位)

DATA2 DCD 3,5,4,-2,0,8,3,-10,5

SUM DCD 0,0,0,0,0,0,0,0,0,0

END

ARM 汇编程序示例

AERA EXAMPLE, CODE, READONLY ; 定义代码段

ENTRY

CODE32 ;声明32位ARM汇编指令

START LDR R1, =DATA1 ;数组DATA1首地址送给R1

LDR R2, =DATA2 ;数组DATA2首地址送给R2

LDR R3, =SUM ;数组SUM首地址送给R3

MOV R0, #0 ;R0充当计数器，初值为0

LOOP LDR R4, [R1], #04 ;取DATA1的一个数，并修改地址指向下一个

LDR R5, [R2], #04 ;取DATA2的一个数，并修改地址指向下一个

ADD~~S~~ R4,R4,R5 ;R4+R5→R4，通过添加“S”来影响标志位

ADD R0, R0, #1 ;R0+1→R0，生成新数组长度

STR R4, [R3], #04 ;保存结果到SUM，并修改地址指向下一个

BNE LOOP ;判断标志位，相加结果不为0则循环

B START ;跳转至START，循环执行

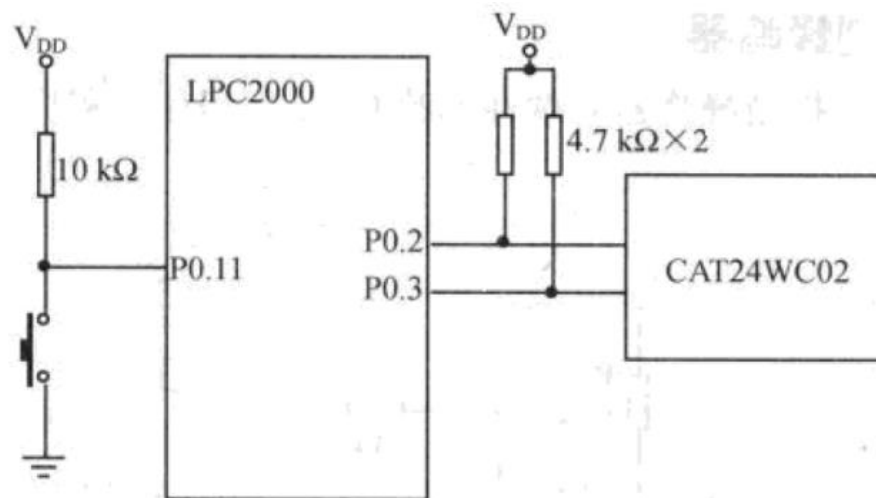
END

ARM系统 GPIO

- **通用**输入输出(GPIO, General Purpose Input/Output)引脚
 - 可以通过编程设置为通用输入、通用输出以及某些特定功能（通过控制寄存器指定和哪个片内的外设管脚连接，如：ADC）
 - GPIO引脚可以与外部设备连接起来，从而实现与外部通讯、控制以及数据采集的功能
- 不同ARM系统有不同的GPIO
 - LPC2000系列最多有4组GPIO端口，可以支持112位IO
 - 如 STM32F103VET6 型号的芯片有 GPIOA~GPIOE 共 5 组 GPIO，芯片一共 100 个引脚，其中GPIO 就占了80个所有的 GPIO 引脚都有基本的输入输出功能

ARM系统 GPIO

- GPIO基本功能
 - 可以独立控制每个GPIO口的方向（输入/输出模式）
 - 可以独立设置每个GPIO的输出状态（高/低电平）： 引脚接入到 LED 灯，就可以控制 LED 灯的亮灭
- 作为输入或者输出使用时，需要接上上拉电阻
 - P0.11——输入； P0.2, P0.3——输出



ARM系统 GPIO

- 控制寄存器（以LPC2000系列为例）
 - 4个端口对应4个控制器组（x=0~3）
 - IOxDIR（方向）：32位 Px.0~Px.31 读写
 - IOxSET（控制输出高电平状态，置位）：32位 Px.0~Px.31
 - IOxCRL（控制输出低电平状态，清零）：32位 Px.0~Px.31
 - IOxPIN（反映引脚电平状态，只读）：32位 Px.0~Px.31

通用名称	功能描述	访问	复位值	控制寄存器组			
				端口 0(P0) 地址 & 名称	端口 1(P1) 地址 & 名称	端口 2(P2) 地址 & 名称	端口 3(P3) 地址 & 名称
IOPIN	反映引脚当前电平状态寄存器	只读	NA	0xE002 8000 IO0PIN	0xE002 8010 IO1PIN	0xE002 8020 IO2PIN	0xE002 8030 IO3PIN
IOSET	GPIO 引脚高电平输出控制寄存器	读/置位	0x0000 0000	0xE002 8004 IO0SET	0xE002 8014 IO1SET	0xE002 8024 IO2SET	0xE002 8034 IO3SET
IODIR	GPIO 引脚输入/输出方向控制寄存器	读/写	0x0000 0000	0xE002 8008 IO0DIR	0xE002 8018 IO1DIR	0xE002 8028 IO2DIR	0xE002 8038 IO3DIR
IOCLR	GPIO 引脚低电平输出控制寄存器	只清零	0x0000 0000	0xE002 800C IO0CLR	0xE002 801C IO1CLR	0xE002 802C IO2CLR	0xE002 803C IO3CLR

ARM系统 GPIO

- 强调接口应用，采用C语言编程控制使用
- 通过PINSELx寄存器配置某个引脚为GPIO后，可用IOxDIR控制引脚的输入输出方向

//设置引脚连接模块，将P0.0-P0.7设置为GPIO

PINSEL0 &=0xFFFFF**00**;

IO0DIR=0x000000**FF**; // 设置为输出

IO0SET=0x000000**FF**; //设置输出高电平

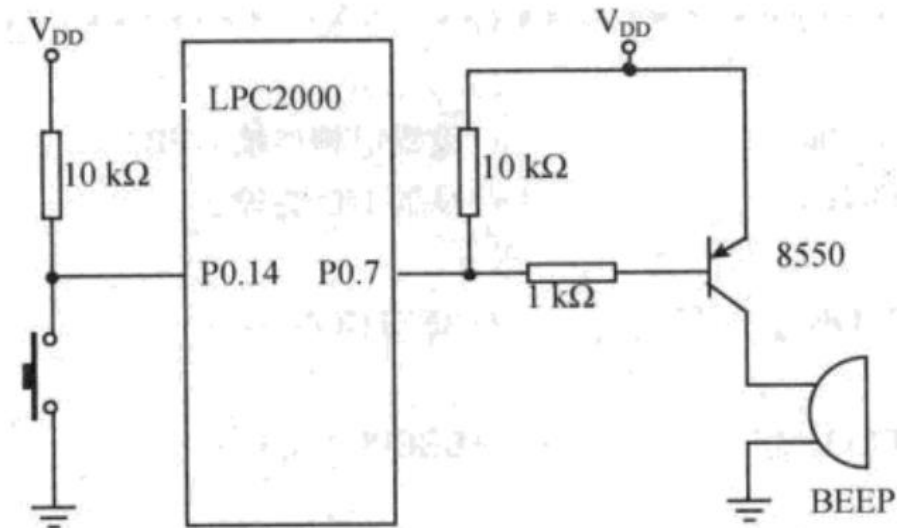
//使作为GPIO功能的P0.0~P0.3输出低电平

IO0CLR=0x0000000**F**; //置位

Bak=IO0PIN; //读取P0端口所有引脚的当前电平状态

ARM系统 GPIO

- 通过判断按键连接的P0.14引脚是否为低电平（开关是否按下）,通过P0.7控制蜂鸣器鸣叫（输出高电平）



```
while(1)
{
    //读取 P0.14引脚是否为0，即按键按下
    (if (( IO0PIN & 0x00004000)!=0)
        IO0SET= 0x00000080 ; //输出高电平，蜂鸣器鸣叫
    else
        IO0CLR= 0x00000080; //输出低电平
    for(i=0; i<1000; i++); //延时
}
return (0);
}
```

```
int main(void)
{
    uint32 i;
    //设置所有引脚为GPIO
    PINSEL0=0x00000000;
    //设置P0.7为输出，其他I/O为输入
    IO0DIR =0x00000080;
```

ARM系统异常与中断

- 中断向量控制器VIC：ARM系统中处理异常的模块
 - VIC接收中断源产生的请求，根据中断号执行对应的中断服务程序

ARM系统中断管理

- 以LPC2000为例（ARM7）
 - 支持32个中断源，不允许中断嵌套
 - CPSR（Current Program Status Register）中标志位可允许/禁止FIQ（具有最高优先级，中断响应最快）、IRQ中断（16个通道，可编辑优先级）
 - 中断选择寄存器（VICIntSelect, 0xFFFF F00C）该寄存器将32个中断请求分别分配为FIQ或IRQ
 - 异常/中断向量表
 - 存放中断向量：中断服务子程序入口地址

ARM系统中断管理

• LPC2000系列 中断向量表

异常类型	中断向量地址
复位	0x00000000
未定义指令	0x00000004
软中断	0x00000008
预取指令终止	0x0000000c
数据访问终止	0x00000010
保留	0x00000014
中断请求 (IRQ)	0x00000018
快速中断请求 (FIQ)	0x0000001c

优先级顺序
复位
数据访问终止
快速中断请求 (FIQ)
中断请求 (IRQ)
预取指令终止
未定义指令
...

ARM系统中断管理

- 异常/中断编号：识别异常/中断源
- 以Cortex架构为例
 - 支持10个系统异常和最多240个外部中断，支持可编程优先级及支持128级抢占（优先级低的中断挂起）
 - 中断编号0~15 在ARM芯片中定义，中断IRQ0~239（编号16-255）由各个芯片商定义

编号	类型	优先级	描述
1	复位	-3	复位
2	NMI	-2	不可屏蔽中断（来自NMI输入脚）
4	存储器管理fault	可编程	访问非法位置
5	总线fault	可编程	总线系统收到错误响应
6	用法fault	可编程	程序错误导致异常
12	调试监视器	可编程	调试监视器（断点等）
16~255	IRQ0~IRQ239	可编程	外部中断#0~#239

ARM系统中断响应

- ARM系统对异常中断的响应过程（以LPC2000为例）
 - ①将寄存器LR(r14)设置为返回地址→X86 返回地址入栈(CS, IP)
 - ②保存当前程序状态寄存器，将CPSR内容保存到异常中断对应的SPSR(Saved Program Status Register)寄存器中→X86 FR入栈
 - ③关中断（设置CPSR）→X86关中断
 - ④查找中断向量表，设置中断向量→X86查找中断向量（4N）
 - ✓发生IRQ中断时，PC指向0x00000018（存放中断程序地址）
 - ✓发生FIQ中断时，PC指向0x0000001C（存放中断程序地址）

ARM系统中断响应

- 中断返回（以LPC2000为例）

①恢复程序状态寄存器，将寄存器SPSR的值复制到CPSR中即可 → X86 FR出栈

②根据LR返回断点 → X86 IP CS出栈

ARM系统中断响应

- 设置中断向量：向量IRQ中断

//中断通道设置为0（IRQ中断），VICIntSelect地址0xFFFF F00C，
固定优先级，通道0中断优先级最高，通道15中断优先级最低

VICIntSelect = 0x00;  优先级设置

//设置中断服务程序地址，VICVectAddr0地址0xFFFF F100

VICVectAddr0 = (unsigned int)Example_ISR;



设置中断向量
(X86)

ARM系统中断响应

- 中断服务子程序

irq会进行保护现场操作

```
void _irq Example_ISR(void)//irq保留字，不同编译器不同
```

```
{
```

```
.....; //中断处理
```

```
VICVectAddr=0x00; //通知VIC中断处理结束
```

```
}
```



清ISR
(X86)

ARM系统中的定时器/计数器

- LPC2000系列：2个32位定时器/计数器
 - 定时器0和定时器1
 - 除了外设基地址不同外，其他功能、结构都相同
- 功能更为丰富：
 - 计数器/定时器
 - 匹配功能：匹配寄存器与读取计数值相同时，可输出高低电平或翻转信号，可用于产生中断
 - 捕获功能：当引脚出现有效信号时，保存当前计数值到捕获寄存器，并产生中断

ARM系统中的定时器/计数器

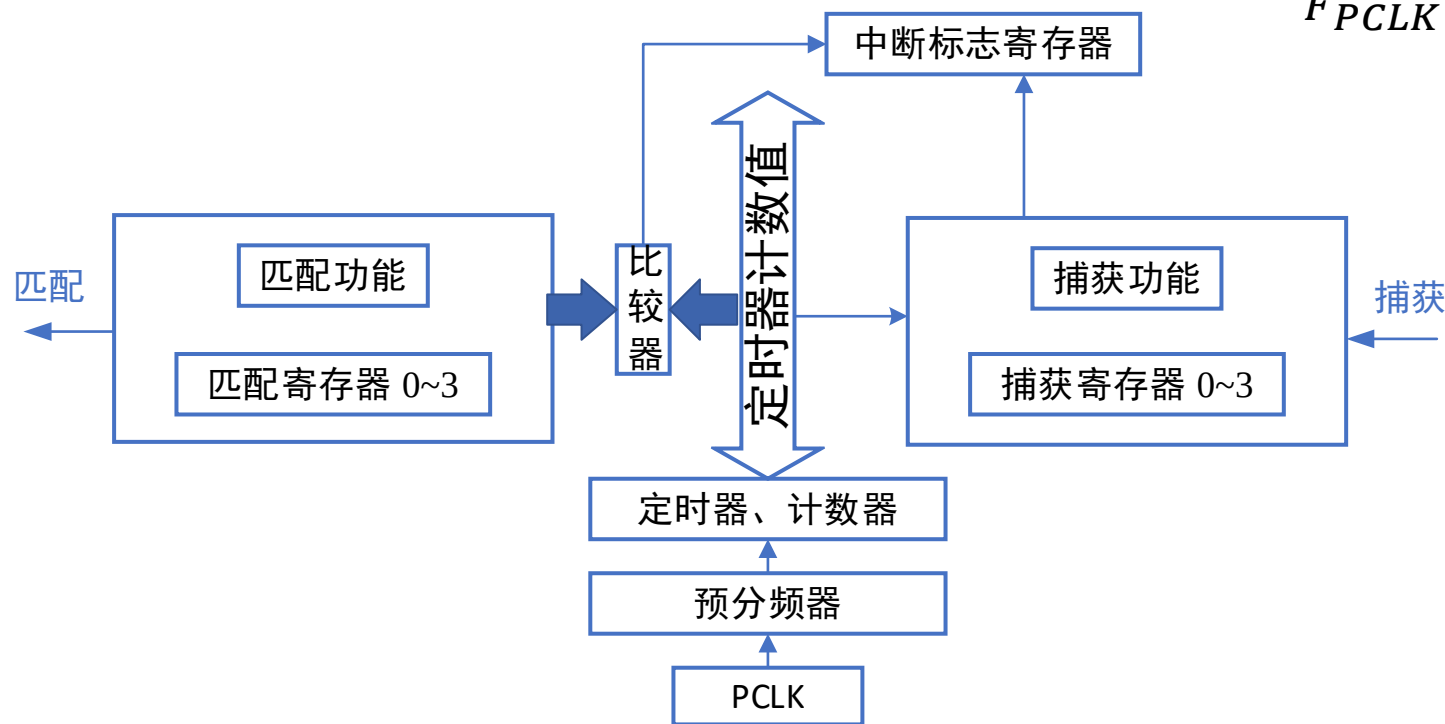
- 管脚

- MAT(match): 4个, 匹配时输出信号
- CAP(capture): 4个, 输入, 捕获引脚的跳变

- 预分频器 (PR预分频寄存器): 预分频后计数时间 $\frac{(PR+1)}{F_{PCLK}}$

预分频相当于增加一个级联功能

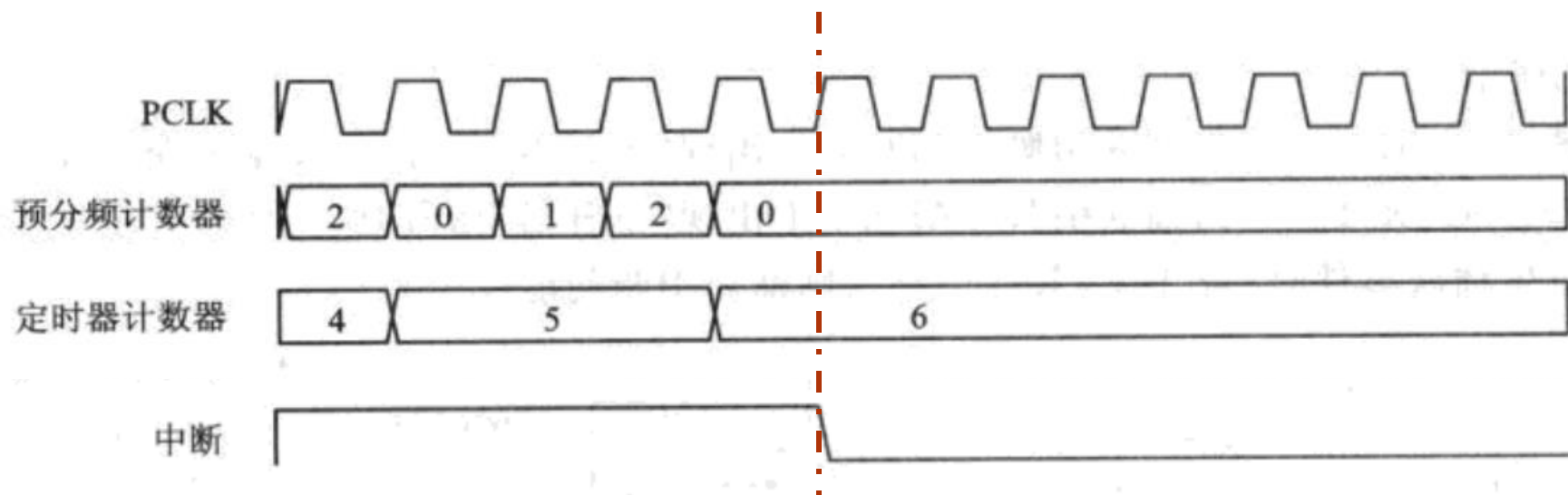
匹配功能: 匹配寄存器与读取计数值相同时, 可输出高低电平或翻转信号, 可用于产生中断



捕获功能: 当引脚出现有效信号时, 保存当前计数值到捕获寄存器, 并产生中断

ARM系统中的定时器/计数器

- 定时中断示例
 - 定时器配置为在匹配时停止计数并产生中断
 - 预分频器（PR寄存器）设置为2，匹配寄存器（MR）设置为6
 - 定时器到达匹配值6的下一个周期，产生中断



ARM系统中的定时器/计数器

- 定时中断示例

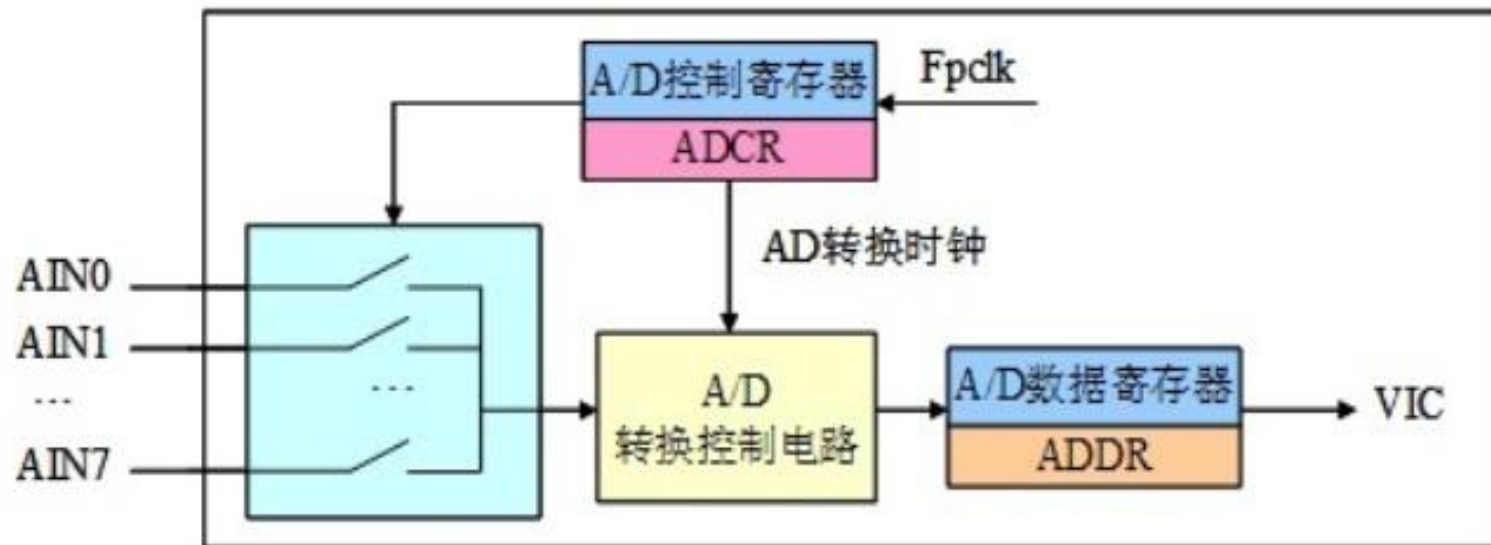
- Time0Init()函数：初始化定时器0，匹配后产生中断，启动定时器

- 定时时间 = $\frac{MR \times (PR + 1)}{F_{PCLK}}$

```
void Time0Init(void)
{
    T0TC=0; //使用定时器0
    T0PR=0; //时钟不分频
    //设置T0MR0匹配后复位T0TC，并产生中断
    T0MCR=0x03;
    T0MR0=10; //设置定时时间10ms
    T0TCR=0x01; //启动定时器0
    /*设置向量中断控制器*/
    .....//定时器0中断分配为IRQ中断
    .....//定时器0中断分配为向量IRQ通道0
    VICVectAddr0=(uint32)Timer0_ISR; //向量IRQ通道0的中断服务程序地址为Timer0_ISR
    .....//定时器0中断使能
}
```


ARM系统ADC

- LPC2200系列具有两个AD转换器
- 具有特性：
 - 10位逐次逼近式模式转换器
 - 测量范围：0~3.3V
 - 10位精度结果需要11个A/D转换时钟，转换时间 $\geq 2.44\mu\text{s}$



ARM系统ADC

- **AD转换触发中断**：当ADC转换结束后会触发中断，ADC处于VIC的通道18，中断使能寄存器VICIntEnable用来控制VIC通道的中断使能
 - 当VICIntEnable[18]=1时，通道18中断使能
- 中断选择寄存器VICIntSelect用来分配VIC通道的中断，VICIntSelect[18]用来控制通道18，即：
 - 当VICIntSelect[18]=1时，ADC中断分配为FIQ中断
 - 当VICIntSelect[18]=0时，ADC中断分配为IRQ中断

ARM系统ADC

- 应用举例
 - AIN0进行10位ADC转换

`PINSEL1=0x00400000;` //配置GPIO寄存器控制引脚P0.27为AIN0功能

`ADCR= (1<<0) |` //SEL=1, 选择通道0

`((Fpclk/1000000-1)<<8) |` //CLKDIV=Fpclk/1000000-1, 即转换时钟频率为1MHz

`(0<<16) |` //BURST=0, 软件控制转换操作

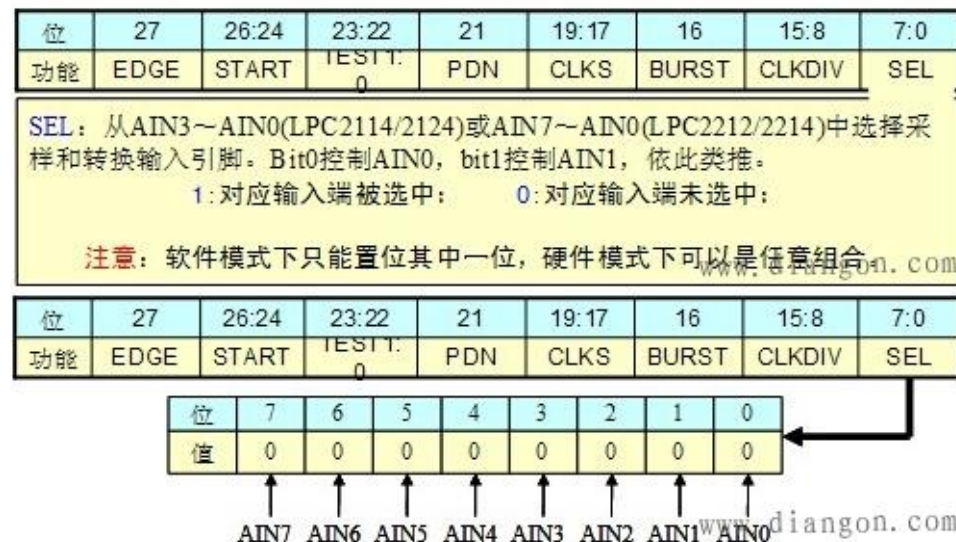
`(0<<17) |` //CLKS=0, 使用11个CLOCK转换

`(1<<21) |` //PDN=1, 正常工作模式

`(0<<22) |` //TEST[1: 0]=00, 正常工作模式

`(1<<24) |` //START=1, 直接启动ADC转换

`(0<<27);` //EDGE=0 (CAP/MAT引脚下降沿触发ADC转换)



设置控制字

ARM系统ADC

- 应用举例（查询方式）
 - 等待转换结束（判断ADDR[31]），再读取结果
 - 由于10位二进制数位于ADDR[15: 6]，因此，需要进行转换

位	31	30	29:27	26:24	23:16	15:6	5:0
功能	DONE	OVERUN	0	CHN	0	V/VddA	0

0：这些位读出为0。用于未来CHN字段的扩展，使之兼容更多通道的转换值。

www.diangon.com

```
uint32 ADC_Data;
```

```
While((ADDR&0x80000000)==0); //等待转换结束ADC_Data=ADDR;
```

```
ADC_Data =(ADC_Data>>6) & 0x3ff; //处理转换值，右移6位后保留后10位
```